Practice Questions on Linear Search and Binary Search

Mixed Practice Questions

Linear Search Practice

Question 1: Basic Linear Search Implementation

```java
public class LinearSearch {
    public static int linearSearch(int[] arr, int target) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                return i;  // Return index if found
            }
        }
        return -1;  // Return -1 if not found
    }

    public static void main(String[] args) {
        int[] numbers = {5, 2, 8, 12, 16, 23, 38, 45};
        int target = 16;
        int result = linearSearch(numbers, target);

        if (result != -1) {
            System.out.println("Element found at index: " + result);
        } else {
            System.out.println("Element not found");
        }
    }
}
```

Question 2: Linear Search with String Array

```java
public class StringLinearSearch {
    public static int searchString(String[] arr, String target) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i].equals(target)) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        String[] fruits = {"apple", "banana", "orange", "grape", "mango"};
        String searchFruit = "orange";
        int index = searchString(fruits, searchFruit);
        System.out.println("Fruit found at index: " + index);
    }
}
```

Question 3: Count Occurrences using Linear Search

```java
public class CountOccurrences {
    public static int countOccurrences(int[] arr, int target) {
        int count = 0;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                count++;
            }
        }
        return count;
    }

    public static void main(String[] args) {
        int[] numbers = {3, 7, 3, 9, 3, 12, 7, 3};
        int target = 3;
        int occurrences = countOccurrences(numbers, target);
        System.out.println("Number " + target + " appears " + occurrences + " times");
    }
}
```

Binary Search Practice

Question 4: Basic Binary Search Implementation

```java
public class BinarySearch {
    public static int binarySearch(int[] arr, int target) {
        int left = 0;
        int right = arr.length - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (arr[mid] == target) {
                return mid;  // Element found
            }

            if (arr[mid] < target) {
                left = mid + 1;  // Search right half
            } else {
                right = mid - 1;  // Search left half
            }
        }
        return -1;  // Element not found
    }

    public static void main(String[] args) {
        int[] sortedArray = {2, 5, 8, 12, 16, 23, 38, 45, 67, 78};
        int target = 23;
        int result = binarySearch(sortedArray, target);

        if (result != -1) {
            System.out.println("Element found at index: " + result);
        } else {
            System.out.println("Element not found");
        }
    }
}
```

Question 5: Recursive Binary Search

```java
public class RecursiveBinarySearch {
    public static int binarySearchRecursive(int[] arr, int target, int left, int right) {
        if (left > right) {
            return -1;  // Base case: element not found
        }

        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            return mid;  // Element found
        }

        if (arr[mid] > target) {
            return binarySearchRecursive(arr, target, left, mid - 1);
        } else {
            return binarySearchRecursive(arr, target, mid + 1, right);
        }
    }

    public static void main(String[] args) {
        int[] sortedArray = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};
        int target = 7;
        int result = binarySearchRecursive(sortedArray, target, 0, sortedArray.length - 1);
        System.out.println("Element found at index: " + result);
    }
}
```

Question 6: Find First Occurrence using Binary Search

```java
public class FirstOccurrence {
    public static int findFirstOccurrence(int[] arr, int target) {
        int left = 0;
        int right = arr.length - 1;
        int result = -1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (arr[mid] == target) {
                result = mid;
                right = mid - 1;  // Continue searching in left half
            } else if (arr[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return result;
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 2, 2, 3, 4, 5, 6};
        int target = 2;
        int firstIndex = findFirstOccurrence(arr, target);
        System.out.println("First occurrence at index: " + firstIndex);
    }
}
```

Comparison Problems

Question 7: Linear vs Binary Search Performance

```java
public class SearchComparison {
    public static void compareSearchMethods() {
        int[] largeArray = new int[1000];
        // Fill array with sorted values
        for (int i = 0; i < largeArray.length; i++) {
            largeArray[i] = i * 2;
        }

        int target = 800;

        // Linear Search
        long startTime = System.nanoTime();
        int linearResult = linearSearch(largeArray, target);
        long linearTime = System.nanoTime() - startTime;

        // Binary Search
        startTime = System.nanoTime();
        int binaryResult = binarySearch(largeArray, target);
        long binaryTime = System.nanoTime() - startTime;

        System.out.println("Linear Search: " + linearTime + " nanoseconds");
        System.out.println("Binary Search: " + binaryTime + " nanoseconds");
    }

    // Include previous search methods here
}
```

Question 8: Search in Rotated Sorted Array

```java
public class RotatedArraySearch {
    public static int searchRotated(int[] nums, int target) {
        int left = 0;
        int right = nums.length - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] == target) {
                return mid;
            }

            // Check if left half is sorted
            if (nums[left] <= nums[mid]) {
                if (target >= nums[left] && target < nums[mid]) {
                    right = mid - 1;
                } else {
                    left = mid + 1;
                }
            } else {
                // Right half is sorted
                if (target > nums[mid] && target <= nums[right]) {
                    left = mid + 1;
                } else {
                    right = mid - 1;
                }
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] rotatedArray = {4, 5, 6, 7, 0, 1, 2};
        int target = 0;
        int result = searchRotated(rotatedArray, target);
        System.out.println("Element found at index: " + result);
    }
}
```

Multiple Choice Quiz Questions

1. What is the time complexity of Linear Search in the worst case?

- A) O(1)
- B) O(log n)
- C) O(n)
- D) $O(n^2)$

Answer: C) O(n)

2. Which condition must be satisfied for Binary Search to work correctly?

- A) Array must contain unique elements
- B) Array must be sorted
- C) Array must have even number of elements
- D) Array must contain only positive numbers

Answer: B) Array must be sorted

3. What does Binary Search return when the target element is not found?

- A) 0
- B) null
- C) -1
- D) false

Answer: C) -1

4. In Binary Search, how is the middle index typically calculated?

- A) mid = (left + right) / 2
- B) mid = left + (right - left) / 2
- C) mid = right - left / 2
- D) Both A and B are correct

Answer: D) Both A and B are correct

5. Which search algorithm is more efficient for searching in a large sorted array?

- A) Linear Search
- B) Binary Search
- C) Both have same efficiency

- D) Depends on the target element position

Answer: B) Binary Search

Practice Coding Questions

1. Implement a method that performs linear search on a 2D array. The method should search for a target value in all rows and columns and return the coordinates (row, column) of the first occurrence.

2. Write a binary search function that finds the last occurrence of a target element in a sorted array containing duplicate values.

3. Create a program that uses binary search to find the square root of a given number (without using built-in sqrt function). The result should be accurate to 2 decimal places.

4. Implement a method that searches for a target string in an array of strings using linear search, but the search should be case-insensitive.

5. Write a binary search algorithm to find the insertion position of a target element in a sorted array. If the element already exists, return its index; otherwise, return the index where it should be inserted to maintain sorted order.